

Digital System Design using VERILOG

Dr. Chandan Karfa

Department of Computer Science and Engineering



भारतीय प्रौद्योगिकी संस्थान गुवाहाटी
INDIAN INSTITUTE OF TECHNOLOGY GUWAHATI



Modeling D-Flip Flop

```
//Asynchronous Reset  
module dff_a (d, clk, rst, q) ;  
    input d, clk, rst ;  
    output q ;  
    reg q ;  
  
    always @ (posedge clk or  
            negedge rst)  
        if (~rst)  
            q <= 1'b0 ;  
        else  
            q <= d ;  
endmodule
```

For active high reset: if (rst)

For active low reset: if (~rst)

Modeling D-Flip Flop

//Synchronous Reset

```
module dff_a (d, clk, rst, q) ;  
  input d, clk, rst ;  
  output q ;  
  reg q ;  
  
  always @ (posedge clk)  
    if (~rst)  
      q <= 1'b0 ;  
    else  
      q <= d ;  
endmodule
```

System Design

Simple Computation (C-like syntax)

...

```
for (x = 0, i = 0; i <= 10; i = i + 1)
```

```
    x = x + y ;
```

```
if (x < 0)
```

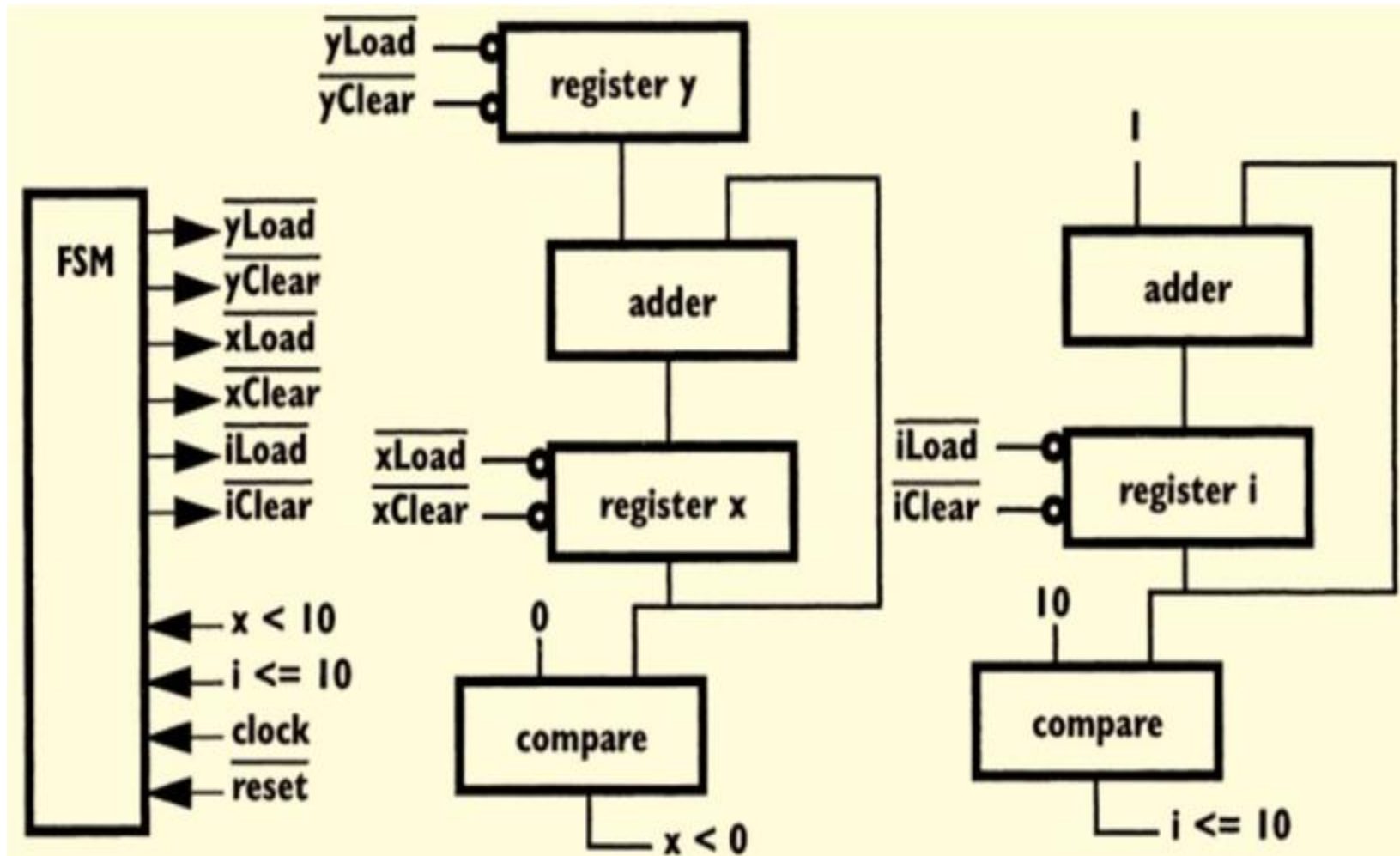
```
    y = 0 ;
```

```
else
```

```
    x = 0 ;
```

...

System Design



for ($x = 0, i = 0; i \leq 10; i = i + 1$)
 $x = x + y$;

If ($x < 0$)
 $y = 0$;
else
 $x = 0$;

Datapath

Datapath modules:

- Register for x, i, y
- Adder
- Comparator

```
module adder
    #(parameter Width = 8)
    (input  [Width-1:0] a,b,
     output [Width-1:0] sum);

    assign sum = a + b;
endmodule
```

```
module register
    #(parameter Width = 8)
    (output reg [Width-1:0] out,
     input  [Width-1:0] in,
     input          clear, load, clock);

    always @(posedge clock)
        if (~clear)
            out <= 0;
        else if (~load)
            out <= in;
endmodule
```

Datapath

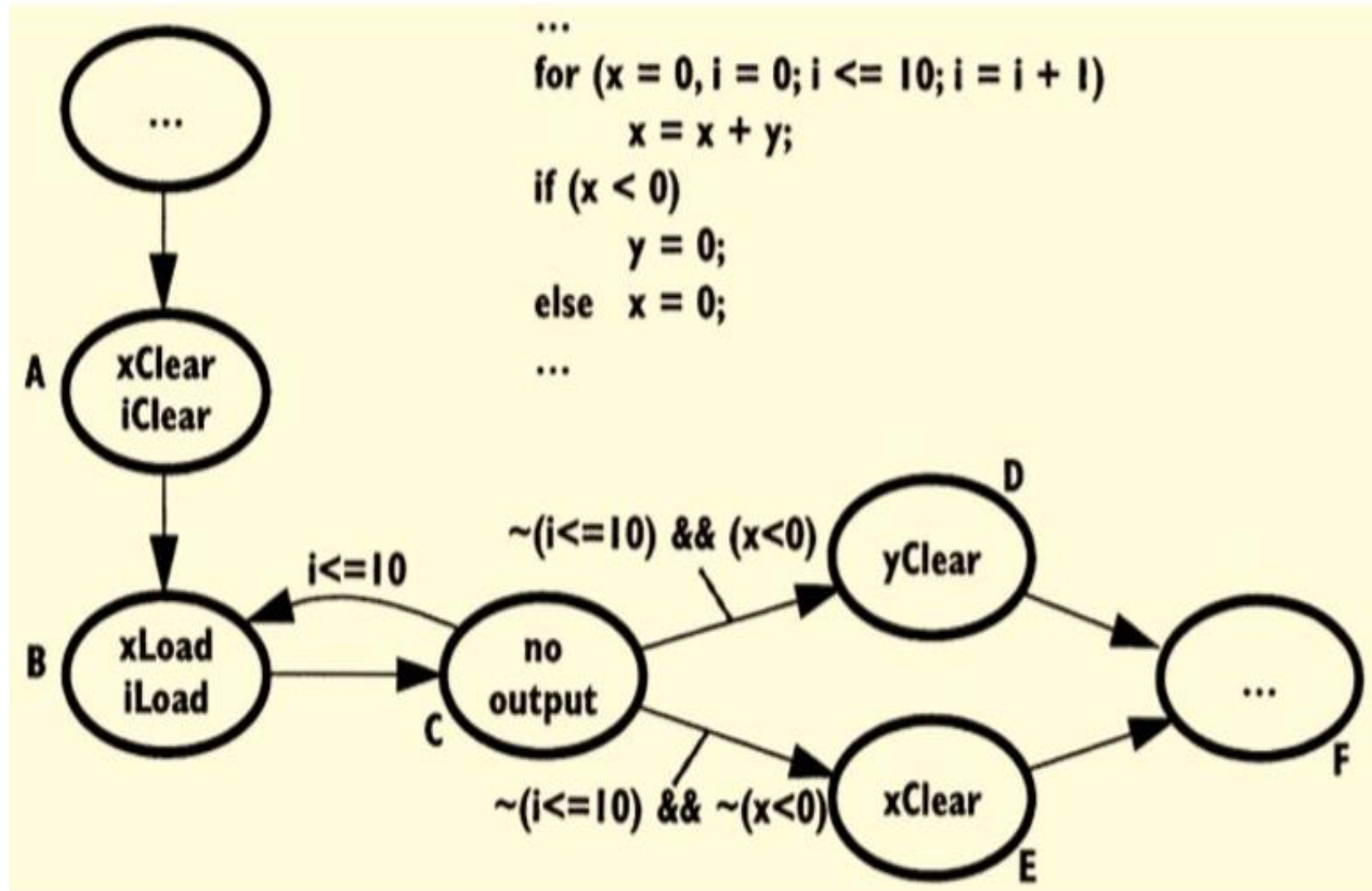
```
module compareLT // compares a < b
    #(parameter Width = 8)
    (input      [Width-1:0]  a, b,
     output                                out);

    assign out = a < b;
endmodule
```

```
module compareLEQ // compares a <= b
    #(parameter      Width = 8)
    (input      [Width-1:0]  a, b,
     output                                out);

    assign out = a <= b;
endmodule
```

Controller FSM

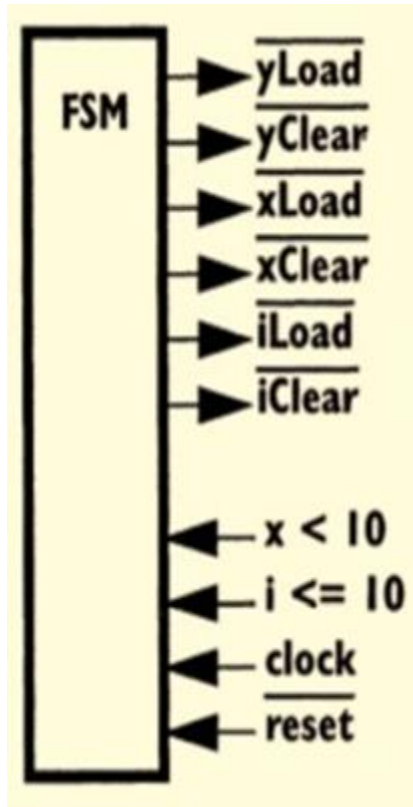



```

module fsm
    (input      LT, LEQ, ck, reset,
     output reg yLoad, yClear, xLoad,
               xClear, iLoad, iClear);

    reg [2:0] cState, nState;

```



```

always @(posedge ck, negedge reset)
    if (~reset)
        cState <= 0;
    else
        cState <= nState;

```

```

always @(cState,LT, LEQ)
    case (cState)
        3'b00:  begin    //state A
                    yLoad = 1; yClear = 1; xLoad = 1; xClear = 0;
                    iLoad = 1; iClear = 0; nState = 3'b001;
                end
        3'b001: begin    // state B
                    yLoad = 1; yClear = 1; xLoad = 0; xClear = 1;
                    iLoad = 0; iClear = 1; nState = 3'b010;
                end
        3'b010: begin    //state C
                    yLoad = 1; yClear = 1; xLoad = 1; xClear = 1;
                    iLoad = 1; iClear = 1;
                    if(LEQ) nState=3'b001;
                    if(~LEQ & LT) nState = 3'b011;
                    if (~LEQ & ~LT) nState = 3'b100;
                end
        3'b011: begin    //state D
                    yLoad = 1; yClear = 0; xLoad = 1; xClear = 1;
                    iLoad = 1; iClear = 1; nState = 3'b101;
                end
        3'b100: begin    //state E
                    yLoad = 1; yClear = 1; xLoad = 1; xClear = 0;
                    iLoad = 1; iClear = 1; nState = 3'b101;
                end
        default: begin // required to satisfy combinational synthesis rules
                    yLoad = 1; yClear = 1; xLoad = 1; xClear = 1;
                    iLoad = 1; iClear = 1; nState = 3'b000;
                    $display ("Oops, unknown state: %b", cState);
                end
    endcase
endmodule

```

Combining the FSM and Datapath

```

module sillyComputation
  #(parameter Width = 8)
  (input      ck, reset,
   input  [Width-1:0] yIn,
   output [Width-1:0] y,x);
  wire  [Width-1:0] i, addiOut, addxOut;
  wire      yLoad, yClear, xLoad, xClear, iLoad, iClear;

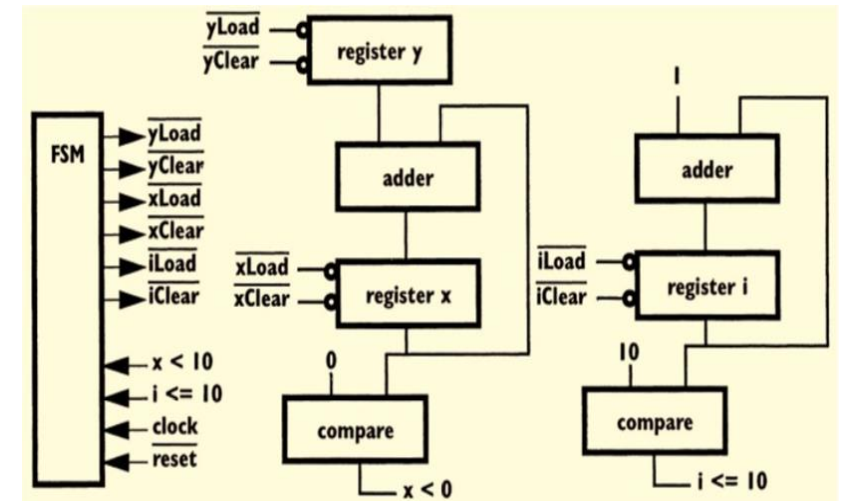
  register      #(Width)  I      (i, addiOut, iClear, iLoad, ck),
                  Y      (y, yIn, yClear, yLoad, ck),
                  X      (x, addxOut, xClear, xLoad, ck);

  adder      #(Width)  addI  (addiOut, 'b1, i),
                  addX  (addxOut, y, x);

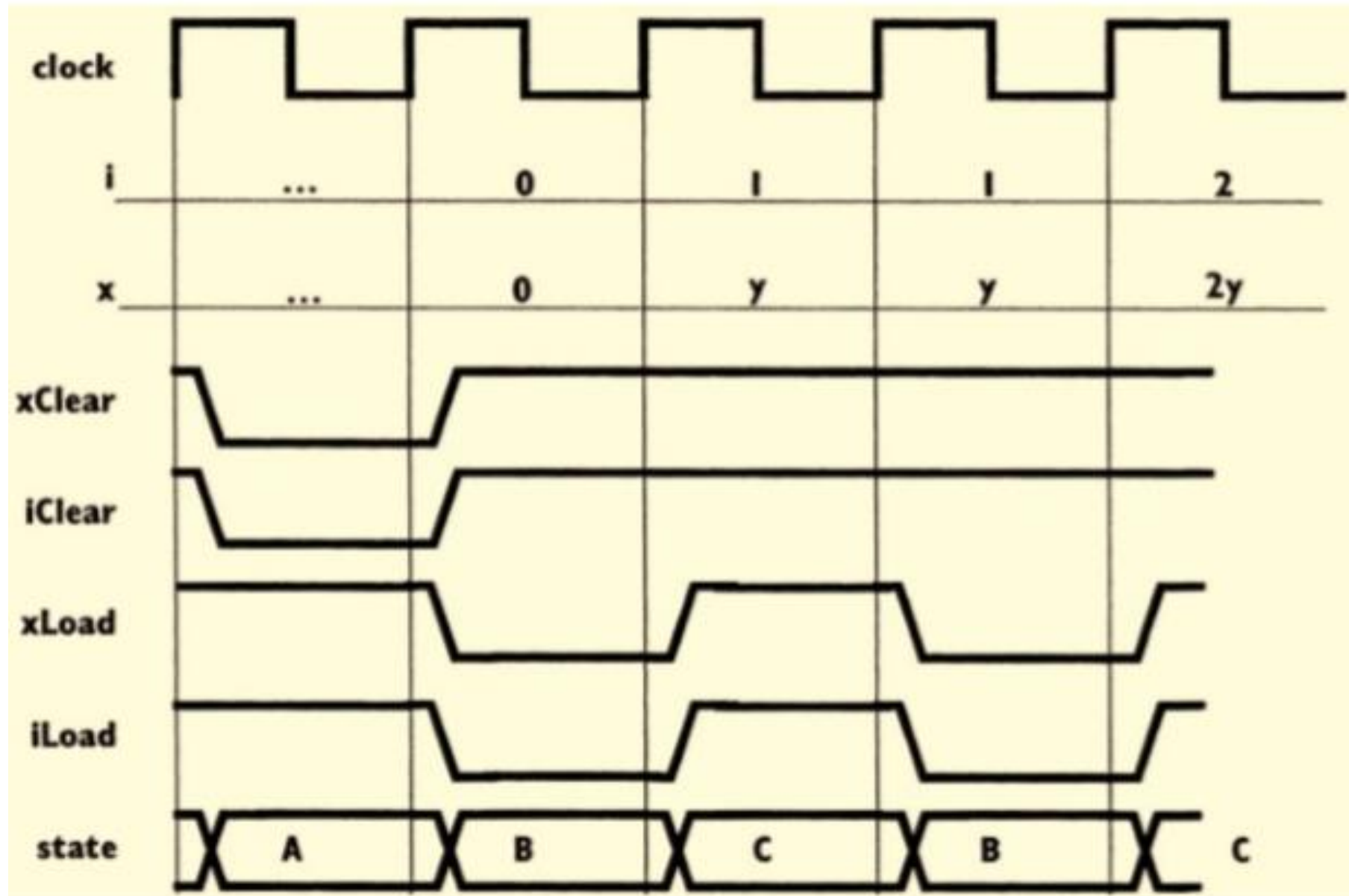
  compareLT   #(Width)      cmpX (x, 'b0, xLT0);
  compareLEQ  #(Width)      cmpI (i, 'd10, iLEQ10);

  fsm          ctl
    (xLT0, iLEQ10, yLoad, yClear, xLoad, xClear, iLoad, iClear, ck, reset);
endmodule

```



Timing Diagram:



Thank You